

C++ & Qt Day4 UDP 통신 및 채팅 구현 보고서

학번: 2026406076

이름: 정창규

제출날짜: 2026.09.08

목차

1. 통신의 기본 개념

- 1.1 통신의 정의
- 1.2 통신의 기본 요소
- 1.3 통신 방식의 분류
- 1.4 통신 프로토콜
- 1.5 유선 통신과 무선 통신

2. UDP 통신

- 2.1 UDP의 개념
- 2.2 OSI 7계층과 UDP
- 2.3 UDP의 특징
- 2.4 UDP 데이터그램
- 2.5 UDP의 장점과 단점
- 2.6 UDP 활용 사례
- 2.7 로봇 시스템에서의 UDP 활용

3. Qt에서의 UDP 통신 구현

- 3.1 Qt Network 모듈
- 3.2 QUdpSocket 클래스
- 3.3 UDP 소켓 생성 및 연결
- 3.4 데이터 수신 방법
- 3.5 데이터 송신 방법

4. 과제 1: UDP 채팅 구현

- 4.1 과제 목표
- 4.2 과제 구성
- 4.3 UI 구성
- 4.4 UDP 통신 구조
- 4.5 메시지 송신 기능 구현
- 4.6 메시지 수신 기능 구현
- 4.7 주요 함수 설명
- 4.8 실행 결과
- 4.9 UDP 송수신 테스트 과정

5. 과제 수행 결과 및 고찰

- 5.1 구현 결과
- 5.2 문제점 및 해결 과정
- 5.3 구현 과정에서 배운 점
- 5.4 느낀 점

1. 통신의 기본 개념

1.1 통신의 정의

통신(Communication)은 두 개 이상의 개체가 서로 데이터를 주고받는 과정을 의미한다. 단순히 데이터를 전달하는 것뿐만 아니라, 전달된 정보를 수신 측에서 올바르게 해석하는 과정까지 포함된다. 따라서 통신이 정상적으로 이루어지기 위해서는 데이터를 보내는 과정과 받은 데이터를 해석하는 과정이 함께 이루어져야 한다. 이번 과제에서 구현하는 UDP 채팅 프로그램 역시 한 컴퓨터에서 입력한 문자열을 다른 컴퓨터로 전송하고, 수신한 내용을 화면에 표시하는 통신 과정으로 볼 수 있다.

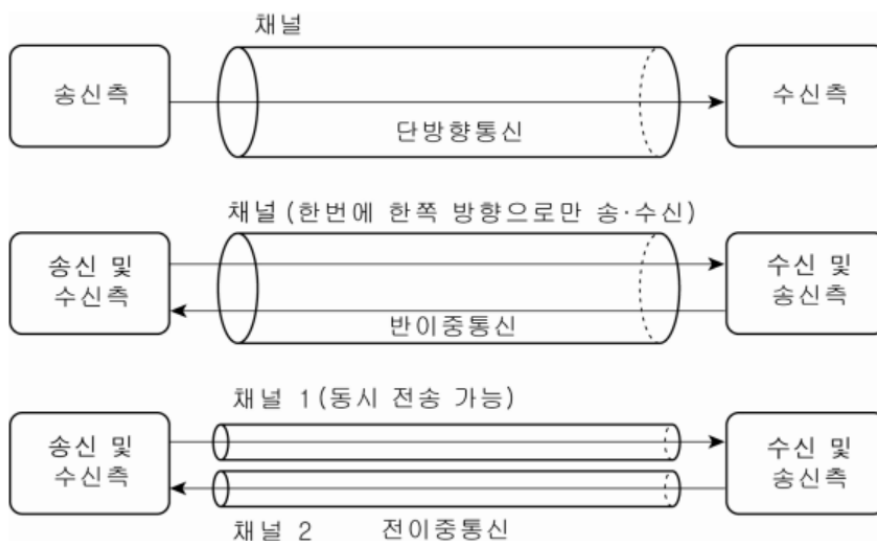
1.2 통신의 기본 요소

통신의 기본 요소에는 송신기(Transmitter), 수신기(Receiver), 전송 매체(Channel), 프로토콜(Protocol)이 있다. 송신기는 데이터를 보내는 역할을 하며, 수신기는 전달된 데이터를 받는 역할을 한다. 전송 매체는 송신기와 수신기 사이에서 데이터가 이동하는 경로이고, 프로토콜은 서로 통신하기 위해 정해진 규칙과 약속을 의미한다. 이번 과제에서는 메시지를 보내는 컴퓨터와 받는 컴퓨터가 각각 송신기와 수신기 역할을 하며, 네트워크를 통해 UDP 방식으로 데이터를 주고받게 된다.

1.3 통신 방식의 분류

통신 방식은 전송 방향에 따라 단방향(Simplex), 반이중(Half Duplex), 전이중(Full Duplex)으로 구분할 수 있다. 단방향은 한쪽에서만 데이터를 전송하는 방식이고, 반이중은 양방향 통신이 가능하지만 동시에 송수신할 수 없다. 전이중은 양쪽에서 동시에 송수신이 가능한 방식이다.

또한 회선 구성 방식에 따라 회선 교환과 패킷 교환으로 나눌 수 있다. 회선 교환은 통신 경로를 연결한 뒤 데이터를 주고받는 방식이며, 패킷 교환은 데이터를 작은 패킷 단위로 나누어 전송하는 방식이다.



1.4 통신 프로토콜

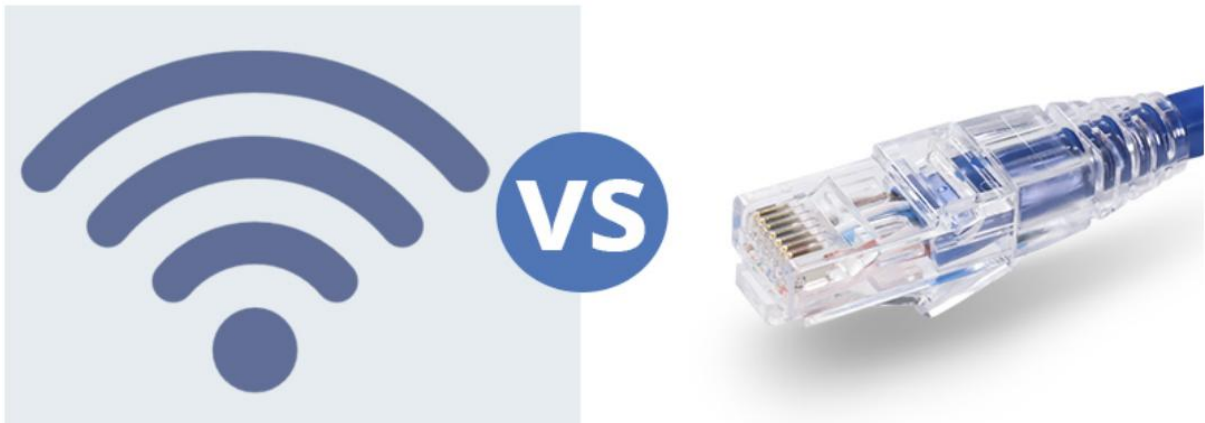
통신 프로토콜(Protocol)은 장치 간 원활한 데이터 통신을 위해 정해 놓은 규칙과 약속을 의미한다. 통신 과정에서는 통신 대상을 구분하고, 데이터 오류를 확인하며, 송수신 속도와 데이터 순서를 맞추는 과정이 필요하다.

주요 기능에는 주소 지정(Addressing), 에러 제어(Error Control), 흐름 제어(Flow Control), 동기화(Synchronization)가 있다. 주소 지정은 통신 대상을 구분하고, 에러 제어는 전송 중 오류를 검출하거나 복구한다. 흐름 제어는 송수신 속도를 조절하며, 동기화는 데이터의 순서와 타이밍을 맞추는 역할을 한다.

1.5 유선 통신과 무선 통신

통신 방식은 전송 매체에 따라 유선 통신과 무선 통신으로 나눌 수 있다. 유선 통신은 LAN 케이블이나 광케이블을 이용하며 비교적 안정적이고 빠른 통신이 가능하다.

무선 통신은 Wi-Fi, Bluetooth, 5G 등을 이용하며 이동성과 편의성이 높다. 통신 방식을 선택할 때에는 속도, 안정성, 범위, 비용 등을 고려해야 한다. 이번 과제에서는 네트워크 환경에서 두 컴퓨터가 UDP로 메시지를 주고받기 때문에 Wi-Fi와 같은 무선 통신을 사용할 수 있다.



2. UDP 통신

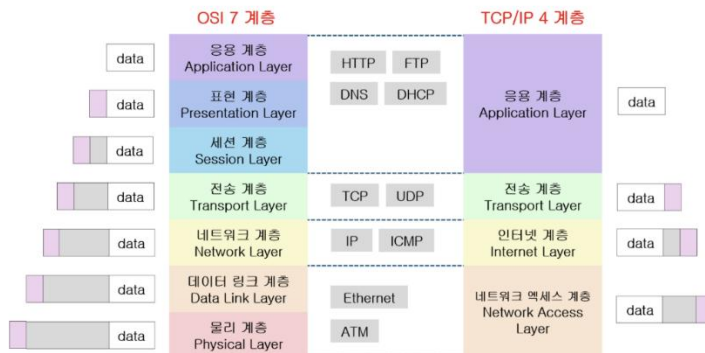
2.1 UDP의 개념

UDP(User Datagram Protocol)는 OSI 7계층 중 전송 계층(Transport Layer)에 속하는 통신 프로토콜이다. 데이터를 전송하기 전에 별도의 연결 과정을 거치지 않는 비연결성 방식을 사용하며, 재전송이나 흐름 제어를 수행하지 않는다. 따라서 데이터의 신뢰성보다는 빠른 전송이 필요한 상황에 적합하다.

UDP는 수신 측과 연결 상태를 확인한 뒤 데이터를 보내는 방식이 아니라, 전송할 데이터가 있으면 바로 목적지로 전달한다. 이 때문에 전송 속도는 빠르지만 데이터가 반드시 도착하거나 순서대로 도착하는 것은 보장하지 않는다.

2.2 OSI 7계층과 UDP

OSI 7계층 모델은 네트워크 통신 과정을 7개의 계층으로 나누어 설명한 구조이다. UDP는 이 중 4계층인 전송 계층(Transport Layer)에 속하며, 전송 계층에서는 종단 간 데이터 전달이 이루어진다. 물리 계층은 신호 전송, 데이터 링크 계층은 MAC 주소 기반의 프레임 전송, 네트워크 계층은 IP 주소 기반의 라우팅을 담당한다. 전송 계층에는 TCP와 UDP가 포함되며, UDP는 IP 계층의 기능을 이용하여 응용 계층에서 전달받은 데이터를 빠르게 전송한다.



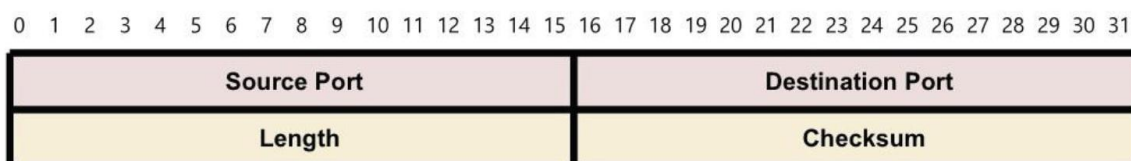
2.3 UDP의 특징

UDP는 데이터를 전송하기 전에 별도의 연결 과정을 거치지 않는 비연결성 통신 방식이다. 수신 측이 준비되어 있는지 확인하지 않고 바로 데이터를 전송할 수 있기 때문에 구조가 단순하고 전송 속도가 빠르다는 특징이 있다. 반면 UDP는 데이터의 도착 여부나 전송 순서를 보장하지 않으며, 손실된 데이터를 다시 보내는 재전송 기능도 제공하지 않는다. 오류 검출은 Checksum을 통해 최소한으로 수행하지만, 오류가 발생한 데이터를 복구하는 기능은 제공하지 않는다. 이러한 특징 때문에 UDP는 신뢰성보다 실시간성과 빠른 전송이 중요한 경우에 적합하며, 영상 스트리밍, 온라인 게임, 센서 데이터 전송 등에 활용된다.

2.4 UDP 데이터그램

UDP에서 데이터를 전송할 때 사용하는 단위를 데이터그램(Datagram)이라고 한다. UDP는 별도의 연결 과정을 거치지 않고 데이터그램 단위로 데이터를 전송하며, 수신 측이 준비되어 있지 않아도 송신할 수 있다. UDP 헤더의 크기는 고정된 8바이트이며, 출발지 포트(Source Port), 목적지 포트(Destination Port), 길이(Length), 체크섬(Checksum)으로 구성된다. 헤더 구조가 단순하기 때문에 처리 과정이 비교적 간단하고 빠른 전송에 적합하다.

UDP Header



2.5 UDP의 장점과 단점

UDP의 가장 큰 장점은 연결 설정 과정이 없고 헤더 구조가 단순하여 빠르게 데이터를 전송할 수 있다는 점이다. 또한 데이터의 재전송이나 흐름 제어를 수행하지 않기 때문에 실시간성이 중요한 통신에 적합하다. 반면 데이터가 목적지에 정상적으로 도착했는지 보장하지 않으며, 데이터의 순서도 보장하지 않는다. 전송 중 손실이 발생하더라도 자동으로 재전송하지 않고, 오류를 검출해도 복구 기능은 제공하지 않는다. 따라서 중요한 데이터를 정확하게 전달해야 하는 경우에는 UDP만 사용하는 것이 적합하지 않을 수 있다.

UDP는 이러한 특징 때문에 영상 스트리밍, 온라인 게임, 센서 데이터 전송처럼 일부 데이터 손실보다 빠른 전달이 더 중요한 환경에서 주로 사용된다.

2.6 UDP 활용 사례

UDP는 빠른 전송이 중요하고 일부 데이터 손실이 발생해도 전체 동작에 큰 영향을 주지 않는 환경에서 주로 사용된다. 대표적인 예로 영상 스트리밍, 온라인 게임, 센서 데이터 전송 등이 있다.

특히 로봇 영상 데이터의 경우 초당 15~30fps 정도로 많은 이미지 데이터를 지속적으로 전송해야 하므로 빠른 통신이 중요하다. 일부 프레임이 손실되더라도 다음 프레임으로 대체할 수 있기 때문에 UDP 방식이 적합하다.

반면 상태 메시지나 명령어처럼 중요한 데이터는 손실되면 문제가 될 수 있으므로, UDP에 CRC8과 같은 오류 검출 방식을 추가하거나 TCP를 사용하는 방식으로 보완할 수 있다.

2.7 로봇 시스템에서의 UDP 활용

로봇 시스템에서는 카메라 영상이나 센서 데이터처럼 짧은 시간 안에 많은 데이터를 계속 전송해야 하는 경우가 많다. 특히 영상 데이터는 초당 여러 프레임이 연속적으로 전달되기 때문에, 일부 데이터가 손실되더라도 빠르게 다음 데이터를 전송하는 것이 중요하다. 이러한 특성 때문에 UDP는 실시간성이 필요한 로봇 통신에 활용될 수 있다.

반면 로봇의 상태 메시지나 제어 명령처럼 손실되면 문제가 발생할 수 있는 데이터는 신뢰성이 중요하다. 이러한 경우에는 UDP에 CRC8과 같은 오류 검출 방법을 추가하거나 TCP를 사용하는 방식으로 보완할 수 있다.

3.1 Qt Network 모듈

Qt에서 네트워크 통신 기능을 사용하기 위해서는 Qt Network 모듈을 프로젝트에 추가해야 한다. 이번 과제에서는 UDP 통신을 구현하기 위해 QUdpSocket 클래스를 사용하므로, CMake 파일에서 기존의 Widgets 모듈과 함께 Network 모듈을 추가하였다.

CMake에서는 find_package()를 통해 Network 모듈을 불러오고, target_link_libraries()에 Network 모듈을 연결하여 네트워크 관련 클래스를 사용할 수 있도록 설정하였다. 이를 통해 이후 UDP 소켓 생성과 데이터 송수신 기능을 구현할 수 있다.

이번 과제에서는 Day3에서 구현한 천지인 키보드 UI를 활용하고, 입력된 문자열을 UDP를 통해 전송하는 방식으로 기능을 확장하였다.

3.2 QUdpSocket 클래스

Qt에서 UDP 통신을 구현하기 위해서는 QUdpSocket 클래스를 사용할 수 있다. QUdpSocket은 IP 주소와 포트 번호를 이용하여 데이터를 송수신할 수 있도록 제공되는 Qt Network 모듈의 클래스이다. 이번 과제에서는 UDP 통신을 위해 QUdpSocket 객체를 생성하고, **9998번 포트**를 사용하도록 설정하였다.

UDP 데이터가 소켓에 도착하면 readyRead 신호가 발생하며, 이 신호를 수신 함수와 연결하여 데이터를 읽을 수 있다. 이를 통해 상대방이 보낸 메시지가 도착했을 때 자동으로 수신 함수를 실행하도록 구성할 수 있다.

이번 과제에서는 천지인 키보드를 통해 입력한 문자열을 UTF-8 형식으로 변환하여 UDP로 전송하고, 상대방에게서 받은 데이터 역시 문자열로 변환하여 대화 내용에 표시하도록 구성하였다.

3.3 UDP 소켓 생성 및 연결

UDP 통신을 사용하기 위해 QUdpSocket 객체를 생성하고, 데이터를 수신할 포트를 설정하였다. 이번 과제에서는 **9998번 포트**를 사용하였으며, 해당 포트로 들어오는 UDP 데이터를 받을 수 있도록 소켓을 구성하였다.

또한 데이터가 도착했을 때 자동으로 수신 함수가 실행되도록 QUdpSocket의 readyRead 신호를 udpRead() 함수와 연결하였다. PPT에서도 UDP 데이터가 들어왔을 때 readyRead 신호를 수신 함수와 연결하는 방식을 사용하고 있다.

```
udpSocket = new QUdpSocket(this);

udpSocket->bind(QHostAddress::AnyIPv4, 9998);

connect(udpSocket, &QUdpSocket::readyRead,
        this, &MainWindow::udpRead);
```

위 코드에서 bind()를 통해 9998번 포트를 사용하도록 설정하고, connect()를 이용하여 데이터가 수신되면 udpRead() 함수가 실행되도록 하였다.

3.4 데이터 수신 방법

UDP 데이터가 수신되면 readyRead 신호를 통해 udpRead() 함수가 실행된다. 수신 함수에서는 먼저 처리하지 않은 데이터그램이 있는지 확인하고, receiveDatagram()을 이용하여 데이터를 받아온다. PPT에서도 hasPendingDatagrams()와 receiveDatagram()을 이용하는 구조를 제시하고 있다.

수신된 데이터는 QByteArray 형태이므로, 이번 과제에서는 UTF-8 형식의 문자열로 변환하여 채팅 내용에 표시하도록 하였다.

```
void MainWindow::udpRead()
{
    while (udpSocket->hasPendingDatagrams()) {
        QNetworkDatagram datagram =
            udpSocket->receiveDatagram();

        QString text =
            QString::fromUtf8(datagram.data());

        ui->chatTextEdit->append("상대 : " + text);
    }
}
```

위 코드에서 hasPendingDatagrams()는 아직 읽지 않은 데이터가 있는지 확인하고, receiveDatagram()은 실제 데이터그램을 받아오는 역할을 한다. 이후 QString::fromUtf8()을 사용하여 수신 데이터를 문자열로 변환하고 대화 내용에 추가한다.

3.5 데이터 송신 방법

UDP로 데이터를 전송할 때에는 먼저 전송할 문자열을 QByteArray 형태로 변환한 뒤 writeDatagram() 함수를 사용한다. writeDatagram()에는 전송할 데이터와 목적지 IP 주소, 포트 번호를 지정한다. PPT에서도 데이터를 보낼 때 QByteArray 형태의 데이터를 목적지 IP와 포트로 전송하는 방식을 사용하고 있다.

이번 과제에서는 천지인 키보드로 입력한 문자열을 toUtf8()을 이용하여 UTF-8 형식의 바이트 데이터로 변환하였다. 이후 상대방의 IP 주소와 9998번 포트를 지정하여 메시지를 전송하도록 구성하였다.

```
QString text = ui->inputLineEdit->text();

QByteArray data = text.toUtf8();

QHostAddress receiverAddress("상대방 IP");

udpSocket->writeDatagram(
    data,
    receiverAddress,
    udpPort
);
```

위 코드에서 toUtf8()은 입력된 문자열을 UDP로 전송할 수 있는 바이트 데이터로 변환하고,

writeDatagram()은 해당 데이터를 상대방의 IP 주소와 9998번 포트로 전송하는 역할을 한다. 초기 구현에서는 9998번 포트를 사용하였으며, 실제 송수신 테스트 과정에서 다른 UDP 데이터가 수신되는 문제가 발생하여 최종적으로 50000번 포트로 변경하였다.

4. 과제 1: UDP 채팅 구현

4.1 과제 목표

이번 과제의 목표는 Qt와 UDP 통신을 이용하여 두 사용자가 서로 문자열 메시지를 주고받을 수 있는 채팅 기능을 구현하는 것이다. 과제에서는 2인 1조로 서로 대화를 주고받는 문자 통신을 구현하도록 요구하고 있으며, 구현한 코드와 실행 영상을 Git에 제출해야 한다.

이번 과제에서는 Day3에서 구현한 천지인 키보드 UI를 활용하여 메시지를 입력하고, 입력된 문자열을 UTF-8 형식으로 변환한 뒤 UDP를 이용하여 상대방에게 전송하도록 구성하였다. 또한 상대방에게서 수신한 메시지는 대화 내용 영역에 표시하도록 하여 양방향으로 문자를 주고받을 수 있도록 구현하였다.

4.2 과제 구성

이번 과제는 크게 **문자 입력부**, **대화 내용 표시부**, **UDP 송수신부**로 구성하였다. 문자 입력부는 Day3에서 구현한 천지인 키보드 UI를 그대로 활용하여 한글, 영문, 숫자 등을 입력할 수 있도록 하였다.

대화 내용 표시부는 자신이 보낸 메시지와 상대방에게서 받은 메시지를 구분하여 확인할 수 있도록 구성하였다. 입력된 문자열은 전송 시 UTF-8 형식으로 변환되며, UDP를 통해 상대방의 IP 주소와 9998번 포트로 전달된다.

수신 측에서는 같은 9998번 포트를 사용하여 데이터를 받고, 수신된 바이트 데이터를 다시 문자열로 변환한 뒤 대화 내용 영역에 표시하도록 구성하였다. 이를 통해 기존의 천지인 입력 기능에 UDP 통신 기능을 추가하여 서로 문자열 메시지를 주고받을 수 있도록 하였다.

4.3 UI 구성

이번 과제의 UI는 Day3에서 구현한 천지인 키보드 형태를 그대로 활용하였다. 기존의 문자 입력창과 천지인 키보드는 유지하고, 상대방과 주고받은 메시지를 확인할 수 있도록 대화 내용 표시 영역을 추가하였다.

문자 입력은 기존 inputLineEdit를 사용하고, 천지인 키보드에서 입력한 문자열이 해당 영역에 표시되도록 구성하였다. 입력이 완료된 뒤 전송 버튼을 누르면 현재 입력된 문자열을 UDP로 상대방에게 전달하도록 하였다.

대화 내용 영역에는 자신이 보낸 메시지와 상대방이 보낸 메시지를 구분하여 표시하도록 구성하였다. 이를 통해 기존의 천지인 입력 기능을 그대로 유지하면서 UDP 채팅 기능을 추가할 수 있도록 하였다.



4.4 UDP 통신 구조

이번 과제에서는 Day3에서 구현한 천지인 키보드로 입력한 문자열을 UDP를 통해 상대방에게 전달하도록 구성하였다. 송신 측과 수신 측은 모두 9998번 포트를 사용하며, 상대방의 IP 주소를 지정하여 메시지를 전송하도록 하였다.

입력된 문자열은 UTF-8 형식으로 변환한 뒤 전송하고, 수신된 데이터는 다시 문자열로 변환하여 대화 내용 영역에 표시하도록 구성하였다. 이를 통해 기존 천지인 입력 기능에 UDP 기반의 문자 송수신 기능을 추가하였다.

```
천지인 입력
↓
문자열 생성
↓
UTF-8 변환
↓
UDP 전송
↓
상대방 수신
↓
채팅창 표시
```

4.5 메시지 송신 기능 구현

메시지 송신 기능은 기존 Day3 과제에서 사용하던 `pressEnter()` 함수를 수정하여 구현하였다. Day3에서는 Enter 버튼을 누르면 현재 입력된 문자열을 `output.txt` 파일에 저장하였지만, Day4에서는 해당 문자열을 UDP를 통해 상대방에게 전송하도록 동작을 변경하였다.

사용자가 천지인 키보드로 문자를 입력하면 `inputLineEdit`에 현재 문자열이 표시된다. Enter 버튼을 누르면 먼저 `commitCurrent()`를 호출하여 조합 중이던 한글 문자를 확정하고, 이후 `inputLineEdit`에 있는 전체 문자열을 가져오도록 하였다. 문자열이 비어 있는 경우에는 불필요한 데이터가 전송되지 않도록 바로 함수를 종료하도록 구성하였다.

전송할 문자열은 `QString` 형태이기 때문에 UDP로 보내기 전에 `toUtf8()`을 이용하여 `QByteArray` 형태로 변환하였다. 이후 `QHostAddress`에 상대방의 IP 주소를 지정하고, `writeDatagram()`을 이용하여 문자열 데이터와 상대방 IP 주소, 포트 번호를 함께 전달하였다.

```
QString text = ui->inputLineEdit->text();

if (text.isEmpty())
    return;

QByteArray data = text.toUtf8();

udpSocket->writeDatagram(
    data,
    receiverAddress,
    udpPort
);

ui->chatTextEdit->append("나 : " + text);
```

메시지 전송 후에는 자신의 대화 내용 영역에 "나 :" 형식으로 전송한 문자열을 표시하도록 하였다. 이후 `committedText`, `cho`, `jung`, `jong`, `vowelKeys` 값을 초기화하여 다음 문자를 바로 입력할 수 있도록 하였다.

이렇게 구현함으로써 기존 천지인 입력 기능은 그대로 유지하면서, Enter 버튼의 동작만 파일 저장에서 UDP 전송 방식으로 변경할 수 있었다. 기존 코드를 모두 새로 작성하지 않고 필요한 부분만 수정하였기 때문에 Day3에서 구현한 입력 기능과 Day4의 통신 기능을 자연스럽게 연결할 수 있었다.

4.6 메시지 수신 기능 구현

메시지 수신 기능은 `udpRead()` 함수를 이용하여 구성하였다. 상대방이 9998번 포트로 메시지를 전송하면 `readyRead` 신호에 의해 해당 함수가 실행되고, 도착한 데이터그램을 순서대로 확인하도록 하였다.

수신된 데이터는 `receiveDatagram()`으로 가져온 뒤 `QString::fromUtf8()`을 이용하여 문자열로 변환하였다. 이후 변환된 문자열을 `chatTextEdit`에 추가하여 상대방이 보낸 메시지를 확인할 수 있도록 하였다.

```
void MainWindow::udpRead()
{
    while (udpSocket->hasPendingDatagrams()) {
        QNetworkDatagram datagram =
            udpSocket->receiveDatagram();

        QString text =
            QString::fromUtf8(datagram.data());

        ui->chatTextEdit->append("상대 : " + text);
    }
}
```

`while`문을 사용하여 아직 처리되지 않은 데이터그램이 있는 동안 계속 데이터를 읽도록 하였으며, 수신한 메시지 앞에는 "상대 : "를 추가하여 자신이 보낸 메시지와 구분할 수 있도록 하였다.

4.7 주요 함수 설명

이번 과제에서는 기존 천지인 입력 기능에 UDP 송수신 기능을 추가하기 위해 여러 함수를 함께 사용하였다. 각 함수는 문자 입력, 송신, 수신, 화면 표시 등의 역할을 담당한다.

함수	역할
<code>pressEnter()</code>	입력된 문자열을 확인하고 UDP로 전송한다.
<code>udpRead()</code>	상대방이 보낸 UDP 데이터를 수신하고 문자열로 변환한다.
<code>updateInput()</code>	천지인으로 입력 중인 문자열을 입력창에 표시한다.
<code>commitCurrent()</code>	현재 조합 중인 한글 문자를 확정하여 본문에 추가한다.
<code>writeDatagram()</code>	지정한 IP 주소와 포트로 UDP 데이터를 전송한다.
<code>receiveDatagram()</code>	수신된 UDP 데이터그램을 가져온다.

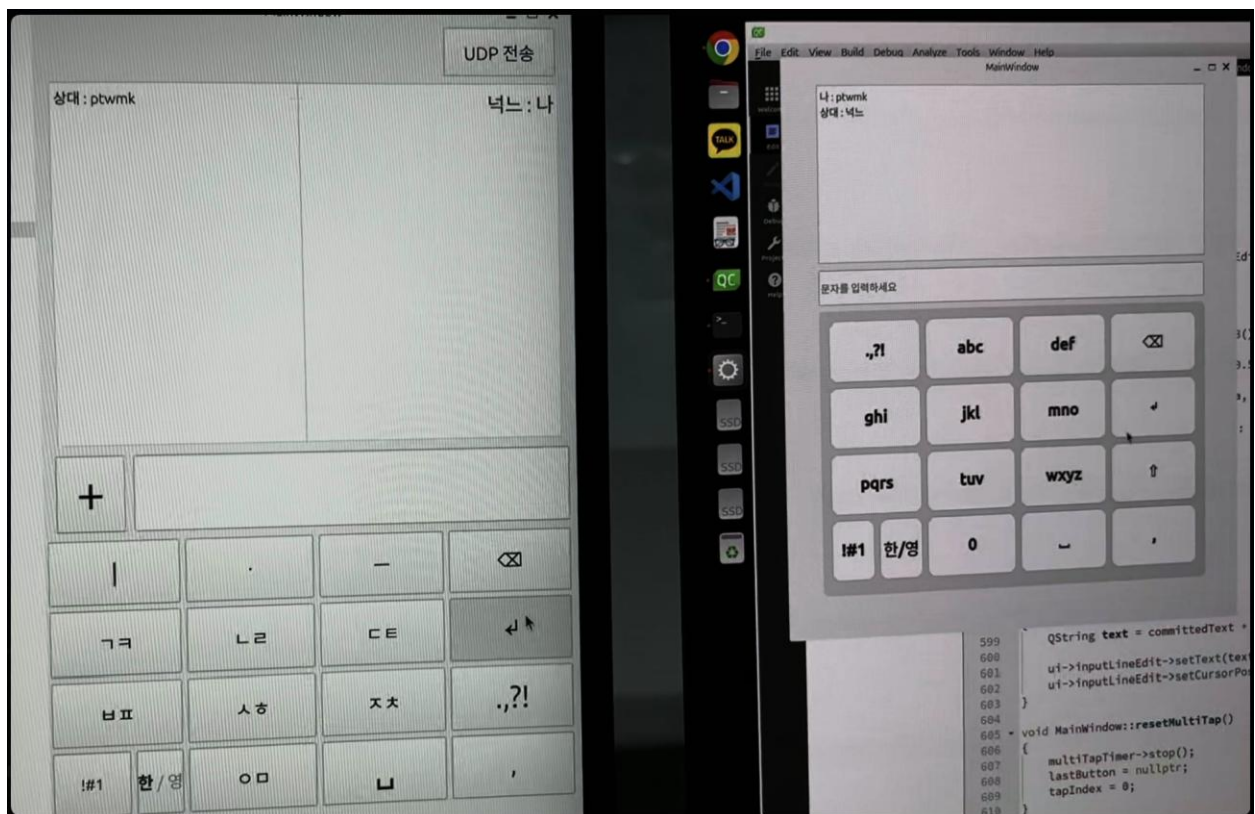
기존 Day3 과제에서 사용하던 `updateInput()`과 `commitCurrent()`는 천지인 입력 기능을 그대로 유지하는 데 사용하였고, Day4 과제에서는 여기에 `pressEnter()`의 UDP 송신 기능과 `udpRead()`의 수신 기능을 추가하여 문자 통신이 가능하도록 확장하였다.

4.8 실행 결과

과제를 실행한 뒤 천지인 키보드를 이용하여 문자열을 입력하고 전송 기능을 확인하였다. 입력한 문자열은 `inputLineEdit`에 표시되며, Enter 버튼을 누르면 해당 문자열이 UDP를 통해 상대방에게 전송되도록 구성하였다.

상대방이 메시지를 수신하면 수신된 문자열이 대화 내용 영역에 "상대 :" 형식으로 표시되고, 자신이 전송한 메시지는 "나 :" 형식으로 표시되도록 하였다. 이를 통해 송신 메시지와 수신 메시지를 구분하여 확인할 수 있도록 하였다.

또한 한글 문자열은 UTF-8 형식으로 변환하여 전송하도록 하였기 때문에 천지인 키보드로 입력한 한글도 정상적으로 송수신할 수 있도록 구성하였다. 포트 번호는 5000번으로 통일하여 두 장치가 동일한 포트를 통해 UDP 데이터를 주고받도록 하였다.



4.9 UDP 송수신 테스트 과정

UDP 통신 기능을 구현한 후 두 장치에서 실제 메시지 송수신 테스트를 진행하였다. 먼저 두 장치가 동일한 네트워크에 연결되어 있는지 확인하고, 각 장치의 IP 주소를 확인하여 상대방의 IP 주소를 송신 코드에 입력하였다. 두 장치에서 사용하는 UDP 포트 번호도 동일하게 설정한 뒤 프로그램을 실행하였다.

메시지 송신 테스트에서는 천지인 키보드를 이용하여 문자열을 입력한 뒤 Enter 버튼을 눌러 상대방에게 전송하였다. 전송한 메시지는 자신의 대화 내용 영역에 "나 :" 형식으로 표시되도록 하였으며, 상대방 프로그램에서는 수신한 메시지가 "상대 :" 형식으로 표시되는지 확인하였다.

또한 반대쪽 장치에서도 동일한 방법으로 메시지를 전송하여 한쪽 방향의 통신뿐만 아니라 양쪽에서 서로 문자를 주고받을 수 있는지 확인하였다. 테스트 결과 한 장치에서 전송한 문자열이 상대방 장치에서 정상적으로 수신되었으며, 상대방이 보낸 문자열 역시 다시 수신되는 것을 확인하였다.

한글 데이터를 전송할 때에는 송신 측에서 문자열을 UTF-8 형식의 바이트 데이터로 변환하고, 수신 측에서 다시 UTF-8 문자열로 변환하도록 하였다. 이를 통해 천지인 키보드에서 입력한 문자를 UDP를 이용하여 전달하는 전체 과정을 확인할 수 있었다.

5. 과제 수행 결과 및 고찰

5.1 구현 결과

이번 과제에서는 Day3에서 구현한 천지인 키보드 UI를 활용하여 UDP 기반의 문자 통신 기능을 추가하였다. 기존의 한글, 영문, 숫자 입력 기능은 그대로 유지하면서 입력된 문자열을 상대방에게 전송하고, 수신된 문자열을 대화 내용 영역에 표시할 수 있도록 구성하였다.

UDP 통신에는 QUdpSocket을 사용하였으며, 송수신 포트는 50000번으로 설정하였다. 입력한 문자열은 UTF-8 형식으로 변환한 뒤 상대방의 IP 주소로 전송하고, 수신한 데이터는 다시 문자열로 변환하여 화면에 표시하도록 구현하였다.

또한 자신이 보낸 메시지와 상대방에게서 받은 메시지를 각각 "나 :"와 "상대 :"로 구분하여 대화 내용을 확인하기 쉽도록 구성하였다. 이를 통해 기존의 천지인 입력 과제에 네트워크 통신 기능을 추가하는 과정을 확인할 수 있었다.

실제 두 장치에서 메시지를 송수신한 결과 한글 문자열이 정상적으로 전달되고 대화 내용 영역에 표시되는 것을 확인하였다.

5.2 문제점 및 해결 과정

실제 UDP 통신 테스트를 진행하면서 예상하지 못한 데이터가 수신되는 문제가 발생하였다. 처음에는 송수신 포트를 9998번으로 설정하여 테스트하였는데, 과제에서 전송하지 않은 XML 형태의 데이터가 대화 내용 영역에 함께 표시되었다.

처음에는 프로그램 코드의 문제라고 생각하였지만, 수신된 데이터의 내용이 과제와 관련이 없는 문자열이었기 때문에 동일한 네트워크에서 다른 프로그램이 같은 UDP 포트를 사용하고 있을 가능성을 확인하였다. UDP는 지정된 포트로 들어오는 데이터그램을 수신하기 때문에 같은 포트를 사용하는 다른 통신 데이터가 들어올 수 있다는 점을 알게 되었다.

이를 해결하기 위해 사용하던 포트 번호를 9998번에서 50000번으로 변경하고, 송신 측과 수신 측 모두 동일한 포트 번호를 사용하도록 다시 설정하였다. 포트 변경 후 프로그램을 다시 실행한

결과 이전에 나타났던 불필요한 XML 데이터가 더 이상 표시되지 않았고, 상대방과 주고받은 문자열만 정상적으로 수신되는 것을 확인하였다.

또한 실제 통신을 위해서는 상대방의 IP 주소를 정확하게 설정해야 했기 때문에 각 장치의 현재 IP 주소를 확인한 뒤 송신 코드에 적용하였다. 이를 통해 UDP 통신에서는 코드 작성뿐만 아니라 IP 주소와 포트 번호를 정확하게 설정하고, 실제 네트워크 환경에서 정상적으로 동작하는지 확인하는 과정도 중요하다는 점을 알 수 있었다.

5.3 구현 과정에서 배운 점

이번 과제를 진행하면서 UDP 통신이 Qt에서 실제로 어떻게 구현되는지 이해할 수 있었다. 이론으로만 보았던 IP 주소와 포트 번호가 실제 코드에서 각각 통신 대상과 데이터 수신 위치를 구분하는 데 사용된다는 점을 확인할 수 있었다.

또한 QUdpSocket을 이용하여 데이터를 송수신할 때 문자열을 바로 전송하는 것이 아니라 QByteArray 형태로 변환해야 한다는 점을 알게 되었다. 특히 한글 데이터를 전송하기 위해 송신 시에는 toUtf8(), 수신 시에는 QString::fromUtf8()을 사용하면서 문자 인코딩의 중요성도 확인할 수 있었다.

기존 Day3에서 구현한 천지인 입력 기능을 그대로 활용하고 UDP 통신 기능을 추가하면서, 기존 코드를 모두 새로 작성하지 않고 필요한 기능만 확장하는 방법도 배울 수 있었다. 이를 통해 UI 입력 기능과 네트워크 통신 기능을 각각 나누어 생각하는 것이 코드를 이해하고 수정하는 데 도움이 된다는 점을 알게 되었다.

5.4 느낀 점

이번 과제를 통해 UDP 통신의 개념을 단순히 이론으로 이해하는 것에서 끝나는 것이 아니라, Qt에서 직접 송수신 기능을 구현하면서 실제 동작 과정을 확인할 수 있었다. 특히 QUdpSocket, IP 주소, 포트 번호가 각각 어떤 역할을 하는지 코드와 함께 확인하면서 UDP 통신의 흐름을 더 쉽게 이해할 수 있었다.

또한 기존 Day3 과제에서 만든 천지인 키보드 UI를 그대로 활용하고 필요한 통신 기능만 추가하면서, 이전에 구현한 기능을 다른 과제에 확장하여 사용할 수 있다는 점도 알게 되었다. 처음부터 모든 기능을 새로 만드는 것보다 기존 코드를 이해하고 필요한 부분만 수정하는 과정이 중요하다고 느꼈다.

이번 과제를 진행하면서 송신과 수신 과정에서 문자열 형식을 맞추는 것과, 상대방의 IP 주소와 포트 번호를 정확하게 설정하는 것이 중요하다는 점도 배울 수 있었다. 이후 다른 네트워크 통신 과제를 진행할 때에도 통신 방식의 특징과 데이터 처리 과정을 먼저 이해한 뒤 구현해야겠다고 생각하였다.